Digital Logic Circuit Design Assignment 2

2025097974 노우림 (Woorim Noh)

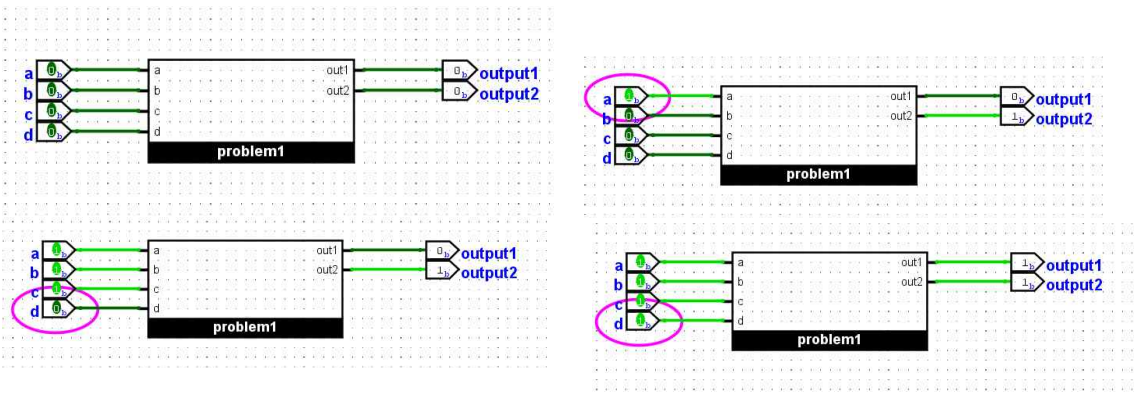
Problem1. VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;

entity problem1 is
    port (
        a, b, c, d : in  std_logic;
        out1, out2 : out std_logic
    );
end problem1;

architecture dataflow of problem1 is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end dataflow;
```

Problem1. Circuit & Simulation Screenshots



Problem1. Explanation of Results

The circuit was simulated with different input combinations. The output out1 became 1 only when all four inputs were 1, while out2 became 1 when at least one input was 1. The results confirmed that the circuit performed the AND and OR operations as expected.

Problem2. VHDL Code

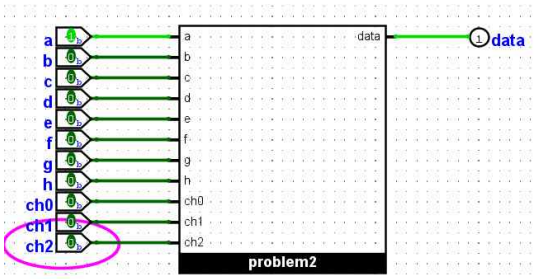
```
library ieee;
use ieee.std_logic_1164.all;

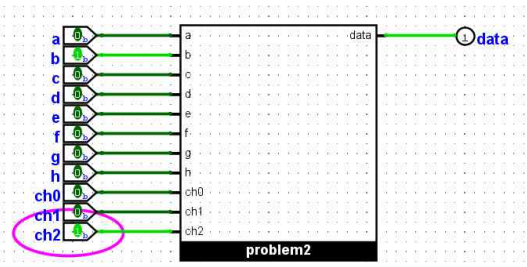
entity problem2 is
    port (
        a, b, c, d, e, f, g, h : in  std_logic;
        ch0, ch1, ch2          : in  std_logic;
        data                   : out std_logic
    );
end problem2;

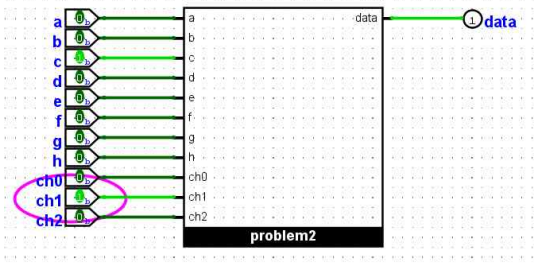
architecture bhv of problem2 is
begin
    process(a, b, c, d, e, f, g, h, ch0, ch1, ch2)
    begin
        if ch0 = '0' then
            if ch1 = '0' then
                if ch2 = '0' then
                    data <= a;
                else
                    data <= b;
                end if;
            else
                if ch2 = '0' then
                    data <= c;
                else
                    data <= d;
                end if;
            end if;
        else
            if ch1 = '0' then
                if ch2 = '0' then
                    data <= e;
                else
                    data <= f;
```

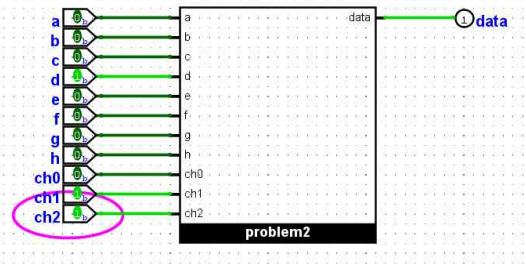
```
            end if;
        else
            if ch2 = '0' then
                data <= g;
            else
                data <= h;
            end if;
        end if;
    end if;
end process;
end bhv;
```

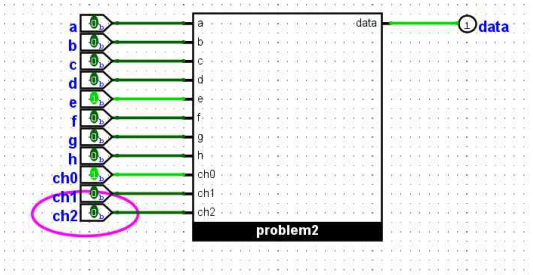
Problem2. Circuit & Simulation Screenshots

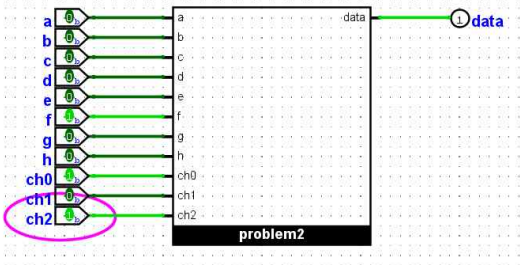


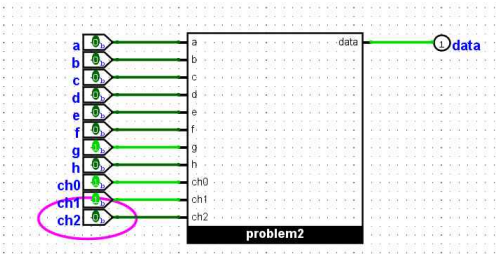


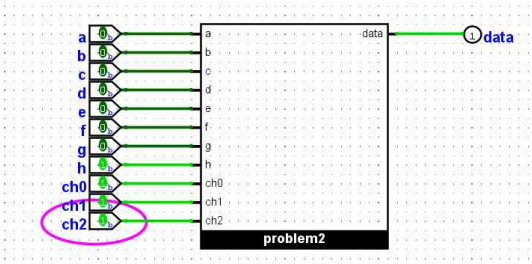












Problem2. Explanation of Results

The 8-to-1 multiplexer was tested with all eight combinations of the select inputs. When ch0 ch1 ch2 was 000, input a appeared at the output data. The other combinations selected the inputs in order: 001 selected b, 010 selected c, 011 selected d, 100 selected e, 101 selected f, 110 selected g, and 111 selected h. In each test, data matched the selected input, confirming that the multiplexer operated correctly.

Problem3. VHDL Code

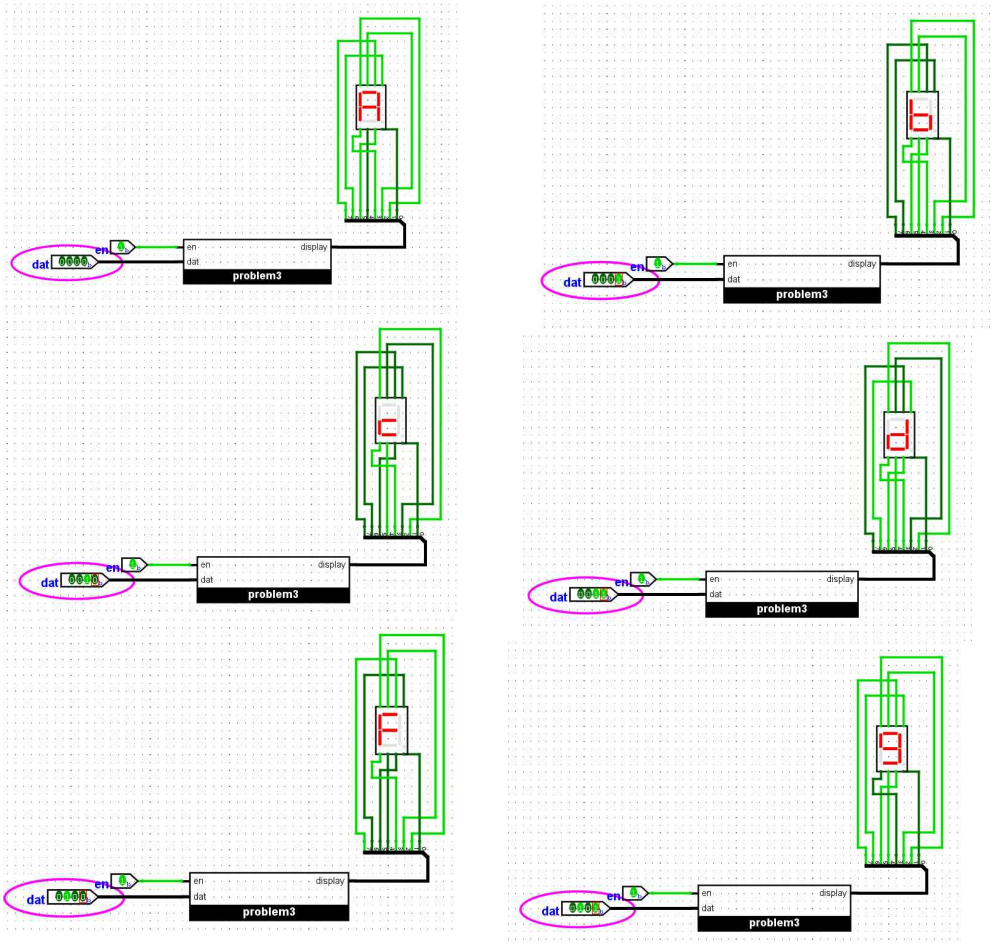
```
library ieee;
use ieee.std_logic_1164.all;

entity problem3 is
    port (
        en      : in  std_logic;
        dat     : in  std_logic_vector(3 downto 0);
        display : out std_logic_vector(7 downto 0)
    );
end problem3;

architecture bhv of problem3 is
begin
    process(en, dat)
    begin
        if en = '0' then
            display <= "ZZZZZZZZ";
        else
            case dat is
                when "0000" => display <= "11101110"; -- a
                when "0001" => display <= "00111110"; -- b
                when "0010" => display <= "00011010"; -- c
                when "0011" => display <= "01111010"; -- d
                when "0100" => display <= "10001110"; -- f
```

```
                when "0101" => display <= "11110110"; -- g
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end bhv;
```

Problem3. Circuit & Simulation Screenshots



Problem3. Explanation of Results

During simulation, en was set to 1 to enable the display. As dat changed from 0000 to 0101, the seven-segment display showed the corresponding characters: a, b, c, d, f, and g. Each input value activated the segment pattern assigned to that character in the VHDL code. When en was set to 0, the display output changed to high impedance (Z), disabling the display output. These results confirm that the circuit responds correctly to both the character input and the enable signal.

Problem4. VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;

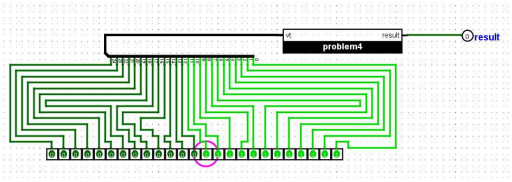
entity problem4 is
    port (
        vt      : in  std_logic_vector(25 downto 1);
        result : out std_logic
    );
end problem4;

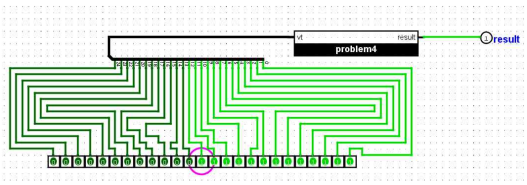
architecture bhv of problem4 is
begin
    process(vt)
        variable sum : integer range 0 to 25;
    begin
        sum := 0;

        for i in 1 to 25 loop
            if vt(i) = '1' then
                sum := sum + 1;
            end if;
        end loop;

        if sum >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end bhv;
```

Problem4. Circuit & Simulation Screenshots





Problem4. Explanation of Results

The simulation results show that the output is 0 when 12 or fewer votes are 1, and changes to 1 when at least 13 votes are 1. This confirms that the 25-person voting machine correctly identifies when a majority of the 25 people vote “yes”.

Problem5. VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;

entity problem5 is
    port (
        vt : in  std_logic_vector(12 downto 0);
        en : in  std_logic;
        a, b, c, d, e, f, g, dp : out std_logic
    );
end problem5;

architecture bhv of problem5 is
begin
    process(vt, en)
        variable count : integer range 0 to 15;
    begin
        if en = '0' then
            a <= '0'; b <= '0'; c <= '0'; d <= '0';
            e <= '0'; f <= '0'; g <= '0'; dp <= '0';
        else
            count := 0;

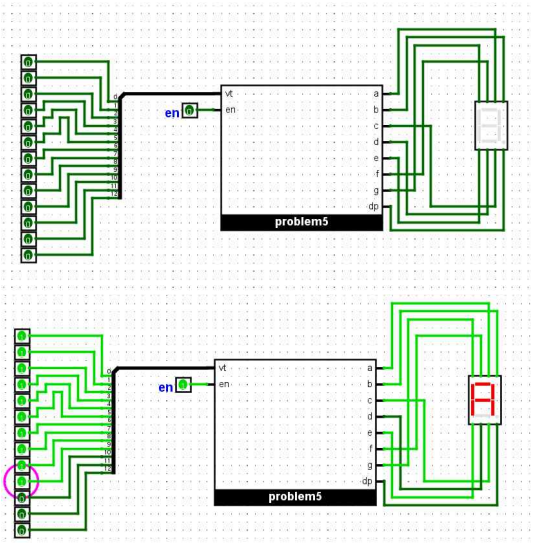
            for i in vt'range loop
                if vt(i) = '1' then
                    count := count + 1;
                end if;
            end loop;

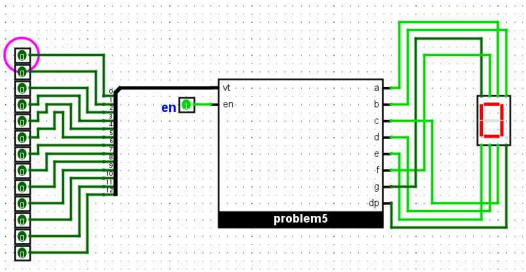
            dp <= '0';

            case count is
                when 0  => a <= '1'; b <= '1'; c <= '1'; d <= '1'; e <= '1'; f <= '1';
g <= '0';
                when 1  => a <= '0'; b <= '1'; c <= '1'; d <= '0'; e <= '0'; f <= '0';
```

```
g <= '0';
                when 2  => a <= '1'; b <= '1'; c <= '0'; d <= '1'; e <= '1'; f <= '0';
g <= '1';
                when 3  => a <= '1'; b <= '1'; c <= '1'; d <= '1'; e <= '0'; f <= '0';
g <= '1';
                when 4  => a <= '0'; b <= '1'; c <= '1'; d <= '0'; e <= '0'; f <= '1';
g <= '1';
                when 5  => a <= '1'; b <= '0'; c <= '1'; d <= '1'; e <= '0'; f <= '1';
g <= '1';
                when 6  => a <= '1'; b <= '0'; c <= '1'; d <= '1'; e <= '1'; f <= '1';
g <= '1';
                when 7  => a <= '1'; b <= '1'; c <= '1'; d <= '0'; e <= '0'; f <= '0';
g <= '0';
                when 8  => a <= '1'; b <= '1'; c <= '1'; d <= '1'; e <= '1'; f <= '1';
g <= '1';
                when 9  => a <= '1'; b <= '1'; c <= '1'; d <= '1'; e <= '0'; f <= '1';
g <= '1';
                when 10 => a <= '1'; b <= '1'; c <= '1'; d <= '0'; e <= '1'; f <= '1';
g <= '1'; -- A
                when 11 => a <= '0'; b <= '0'; c <= '1'; d <= '1'; e <= '1'; f <= '1';
g <= '1'; -- b
                when 12 => a <= '1'; b <= '0'; c <= '0'; d <= '1'; e <= '1'; f <= '1';
g <= '0'; -- C
                when 13 => a <= '0'; b <= '1'; c <= '1'; d <= '1'; e <= '1'; f <= '0';
g <= '1'; -- d
                when 14 => a <= '1'; b <= '0'; c <= '0'; d <= '1'; e <= '1'; f <= '1';
g <= '1'; -- E
                when 15 => a <= '1'; b <= '0'; c <= '0'; d <= '0'; e <= '1'; f <= '1';
g <= '1'; -- F
                when others =>
                    a <= '0'; b <= '0'; c <= '0'; d <= '0';
                    e <= '0'; f <= '0'; g <= '0';
            end case;
        end if;
    end process;
end bhv;
```

Problem5. Circuit & Simulation Screenshots





Problem5. Explanation of Results

The simulation counts the number of bits set to '1' in the 13-bit input vt and displays the corresponding digit or letter when en is '1'. When en is '0', the display turns off. The results show that the output changes according to the input and enable signal, confirming that the module operates correctly.